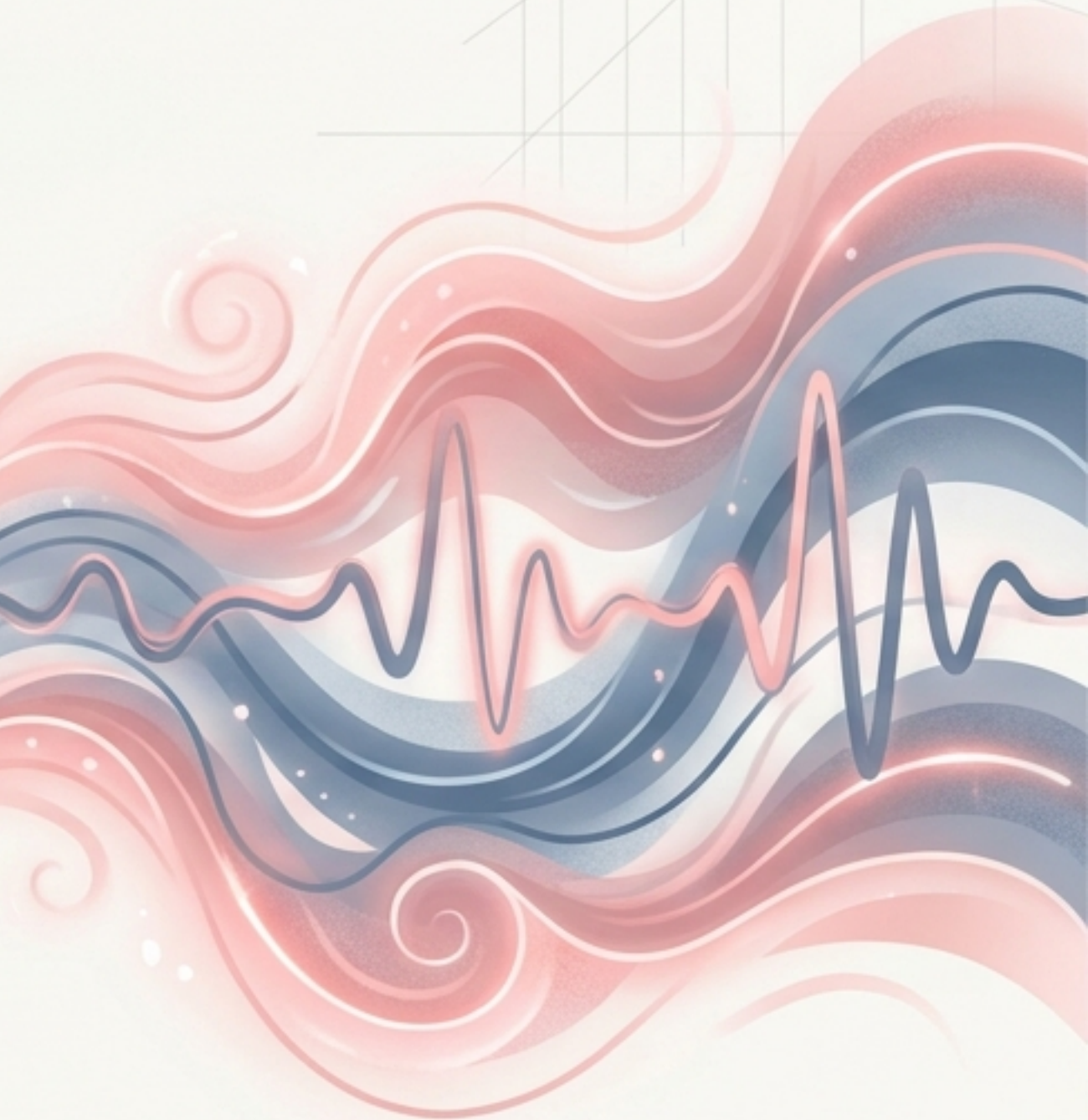
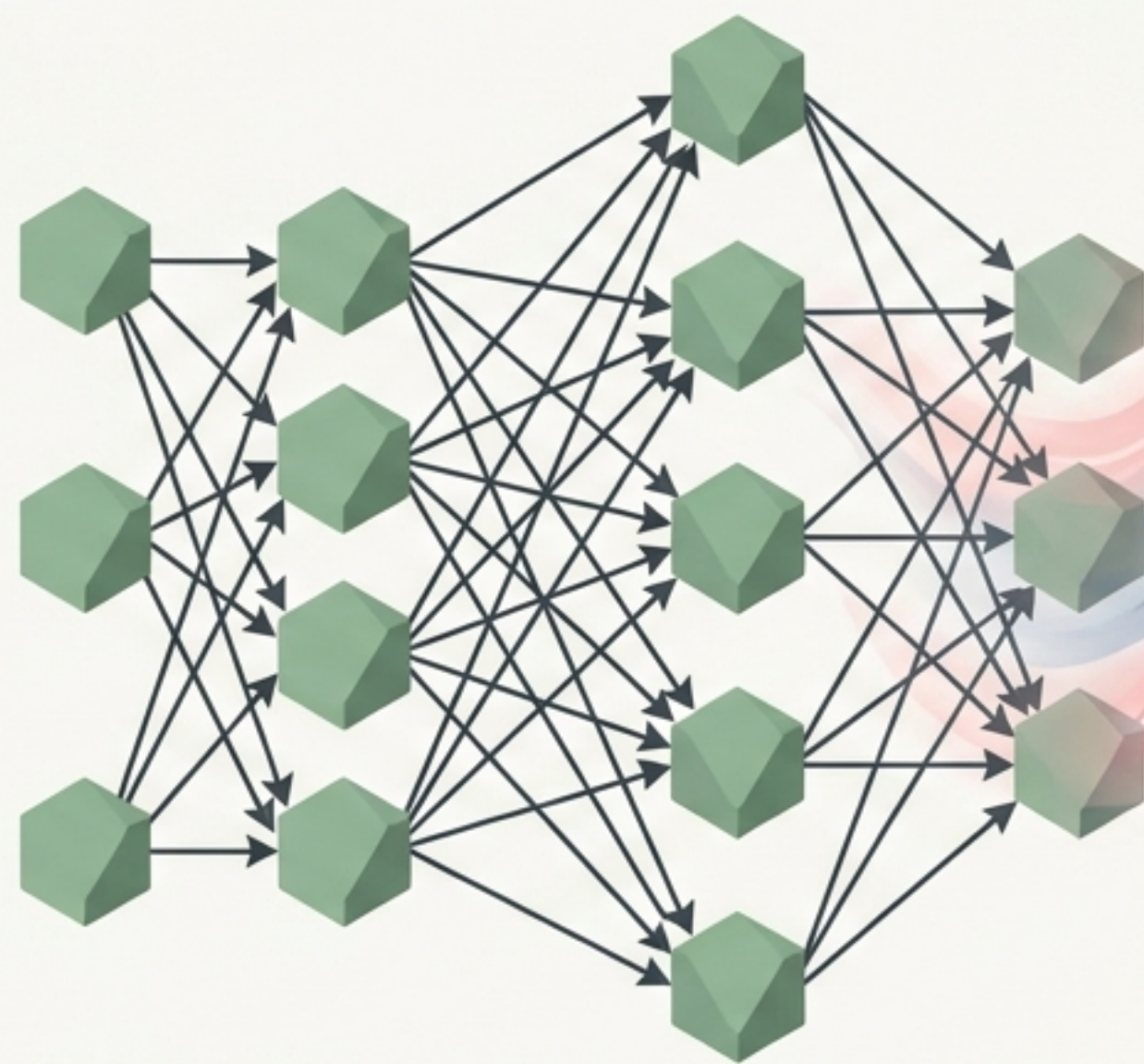


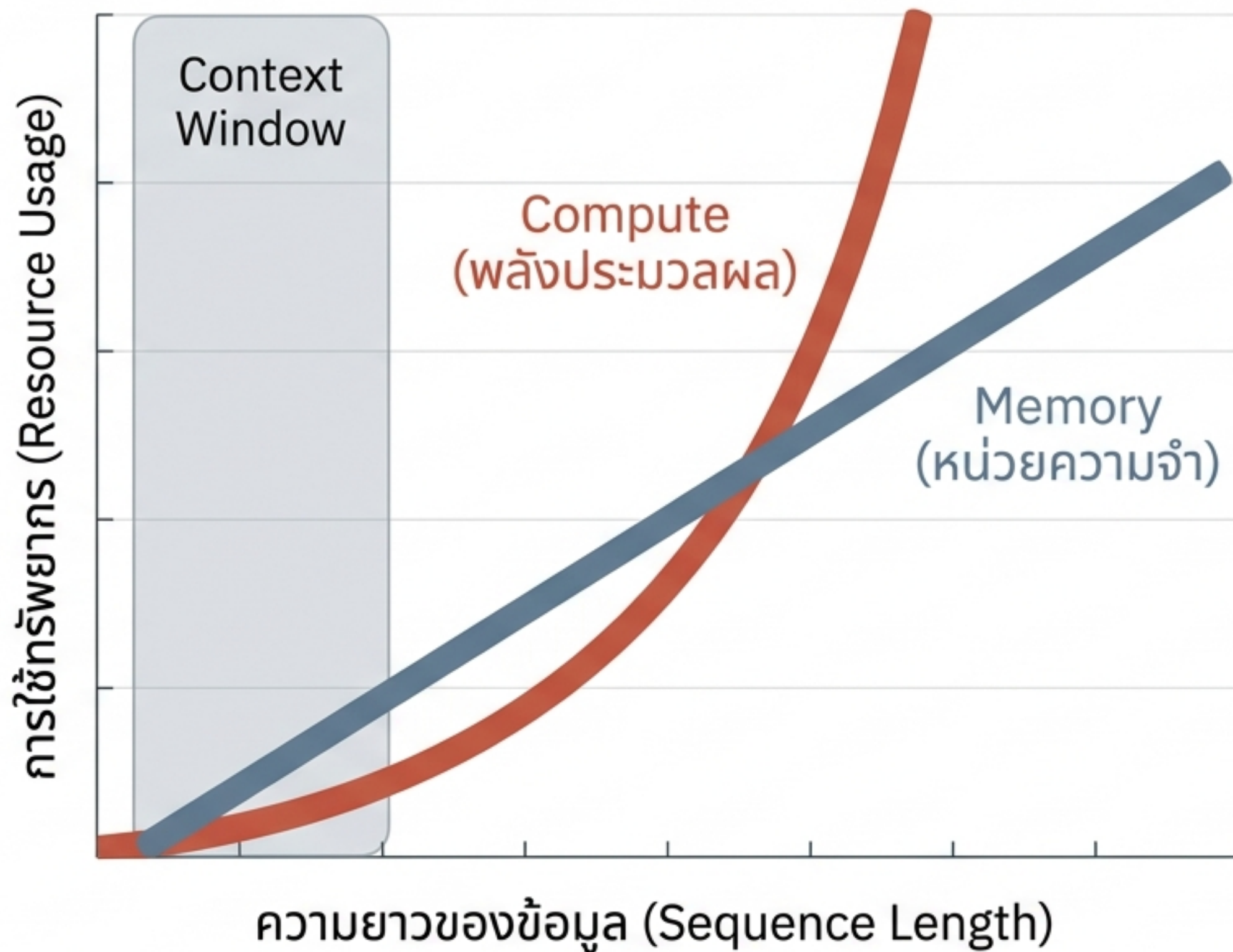
โมเดลภาษาจำเป็นต้อง ‘นอนหลับ’ หรือไม่?

การใช้ Offline Recurrence เพื่อยกระดับความสามารถในการวิเคราะห์เชิงลึก
โดยไม่ลดความเร็วของ Online Inference

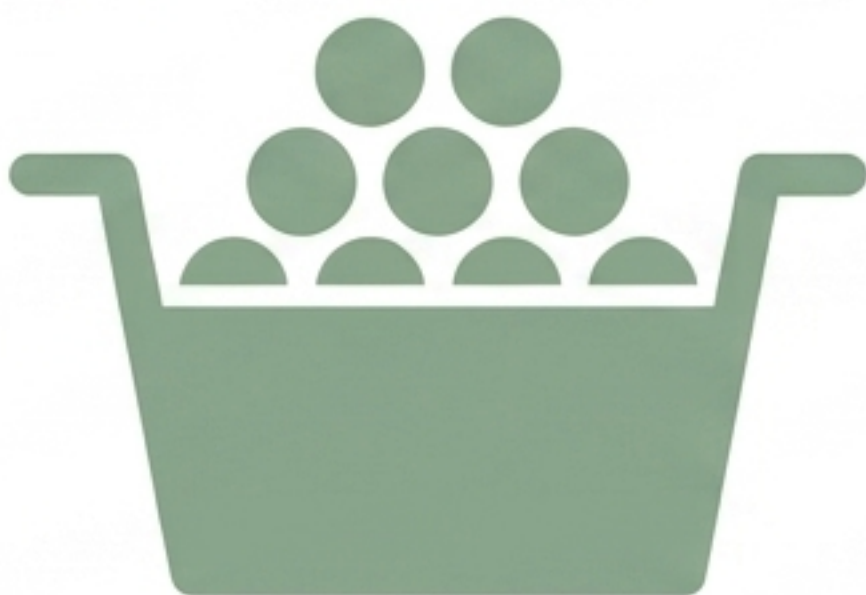


คอขวดของสถาปัตยกรรม Memory ในปัจจุบัน

ปัญหา: เมื่อ Context ยาวขึ้น
โมเดลต้องการทรัพยากรทั้งการ
ประมวลผลและหน่วยความจำ
มหาศาลเพื่อดึงข้อมูลเก่า



ความจุที่เพิ่มขึ้น ไม่ได้แปลว่า ‘คิด’ ได้ลึกขึ้น



ตะกร้า Fast Weights
(เก็บข้อมูลได้)

Eviction Boundary
(การลบทิ้ง)



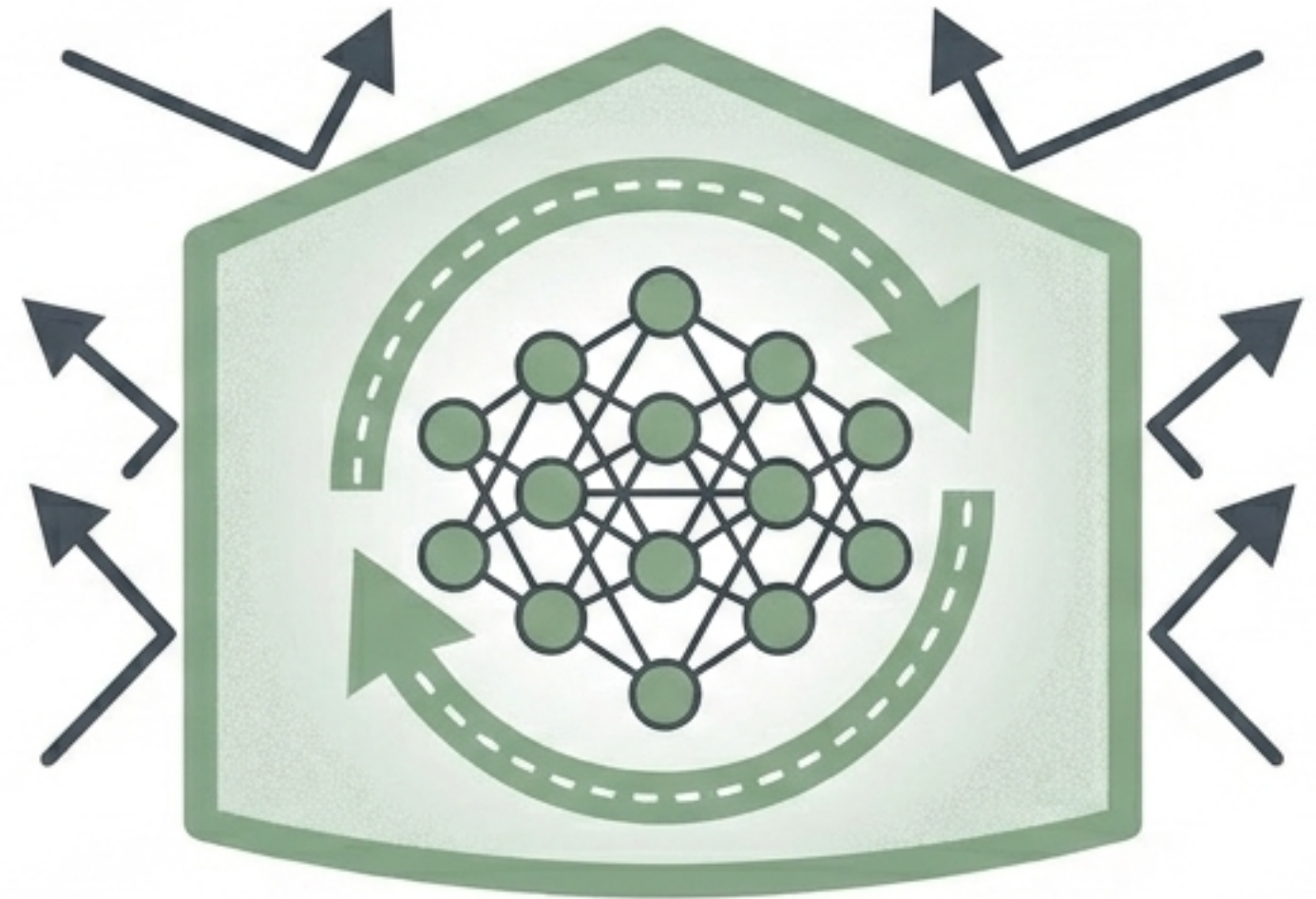
การวิเคราะห์เชิงลึก
(ลึ้มเหลว)

สถาปัตยกรรม Hybrid ปกติ บีบอัดข้อมูลเก่าลงใน Fast Weights ได้...
แต่เมื่อต้องคิดวิเคราะห์โจทย์ที่ซับซ้อนกับข้อมูลที่ถูกลบไปแล้ว
โมเดลกลับทำงานลึ้มเหลว เพราะขาดพลังประมวลผลในการจัดระเบียบ State

แรงบันดาลใจจากธรรมชาติ: การจัดระเบียบความทรงจำ (Memory Consolidation)

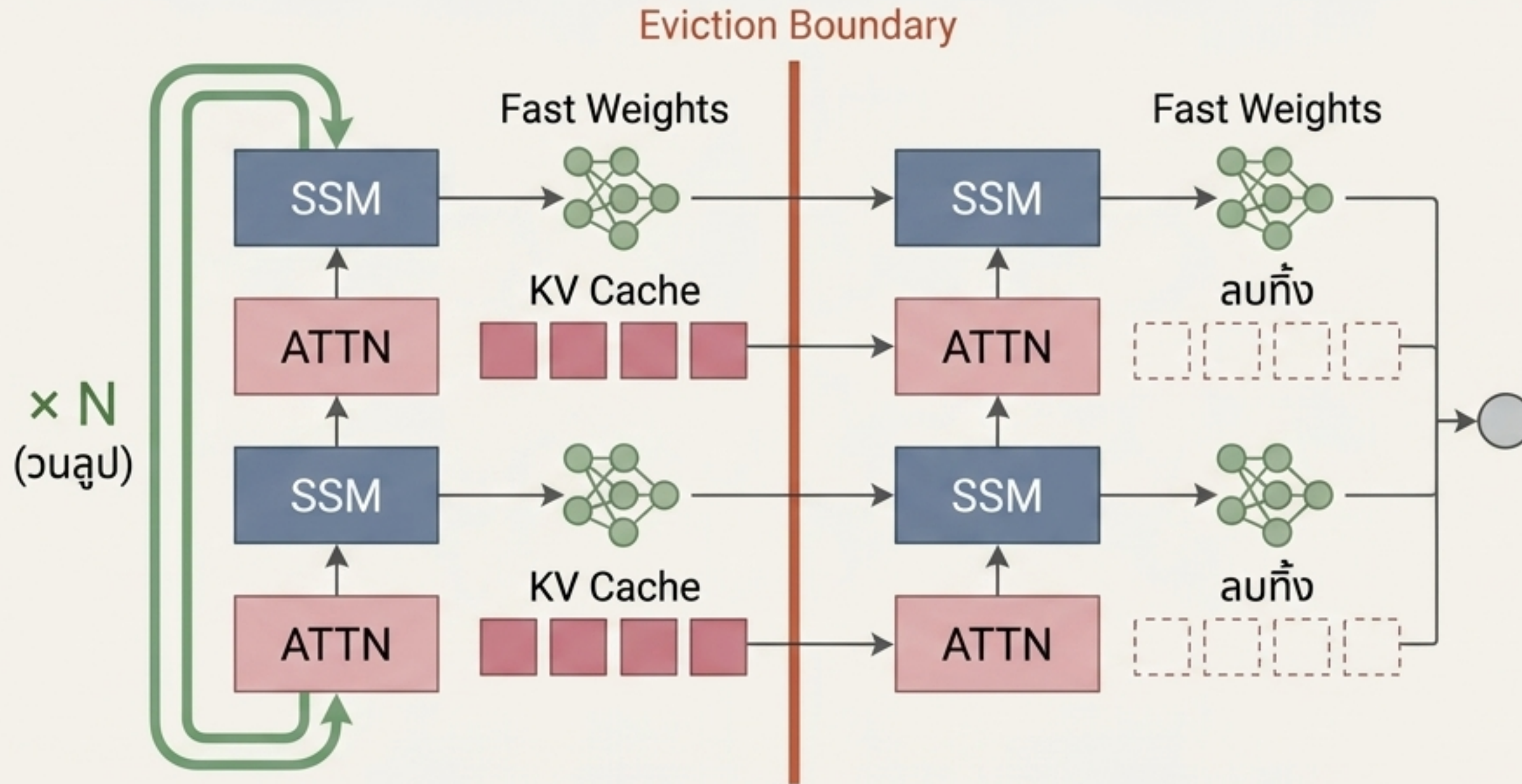


ช่วงนอนหลับ: สัตว์ตัดการรับรู้จากภายนอก เพื่อนำความจำระยะสั้นมากระตุ้นซ้ำ (Replay) และเปลี่ยนเป็นความจำระยะยาว



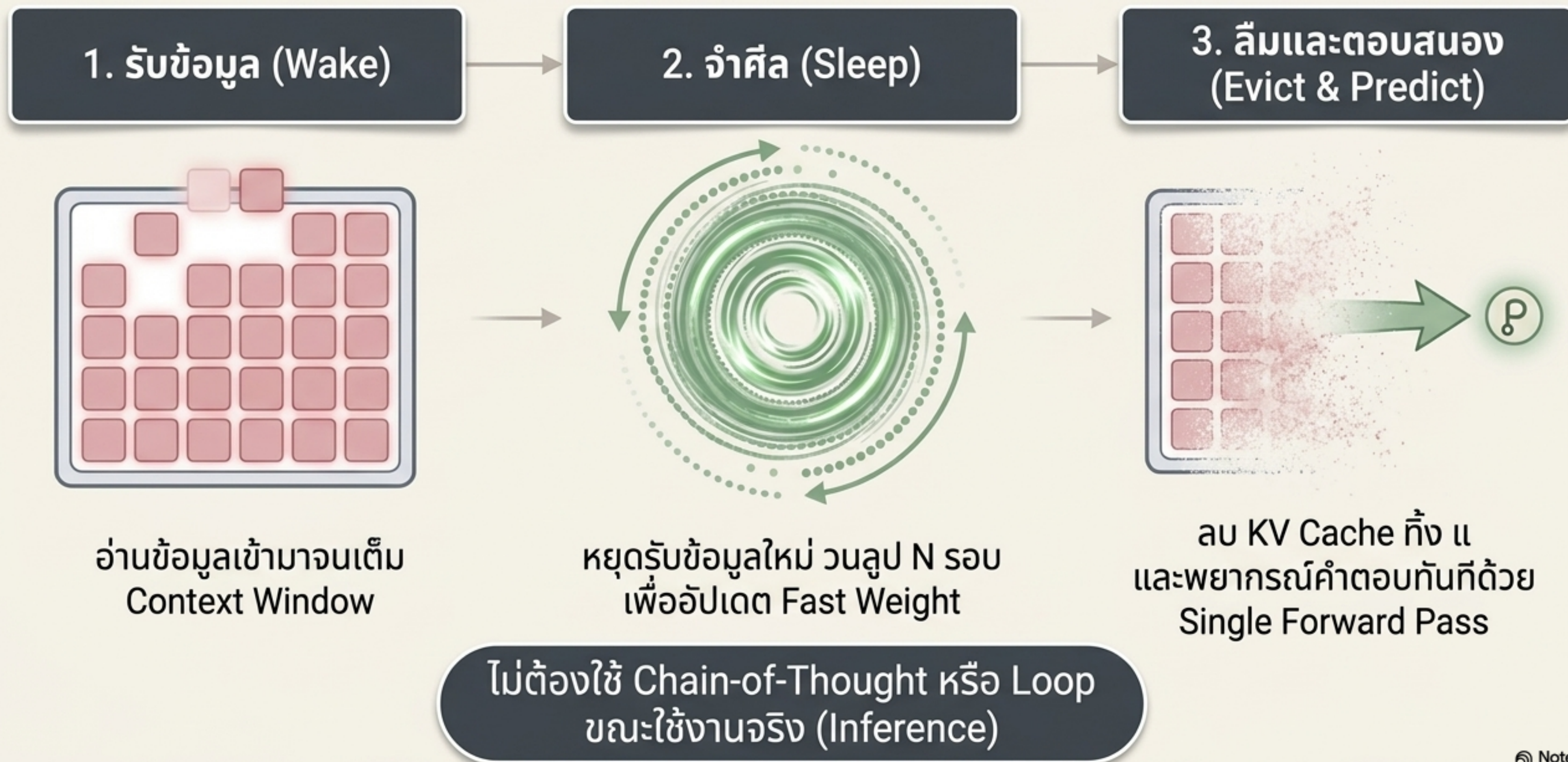
LLM สมัยใหม่: หากเราให้โมเดลหยุดรับ Token ชั่วคราว เพื่อวนลูปประมวลผล จัดระเบียบข้อมูลก่อนลบ KV Cache ทิ้งล่ะ?

สถาปัตยกรรม 'LLM Sleep': ให้โมเดลนอนหลับคิดก่อนลบทิ้ง



ย้ายภาระการคำนวณอันหนักหน่วง
ไปไว้ในช่วงการจัดระเบียบ Fast Weights (Sleep)
เพื่อรักษาความเร็วในการพยากรณ์ตอนต้น

เจาะลึกกลไก: วงจรการตื่นและหลับ (Wake-Sleep Cycle)



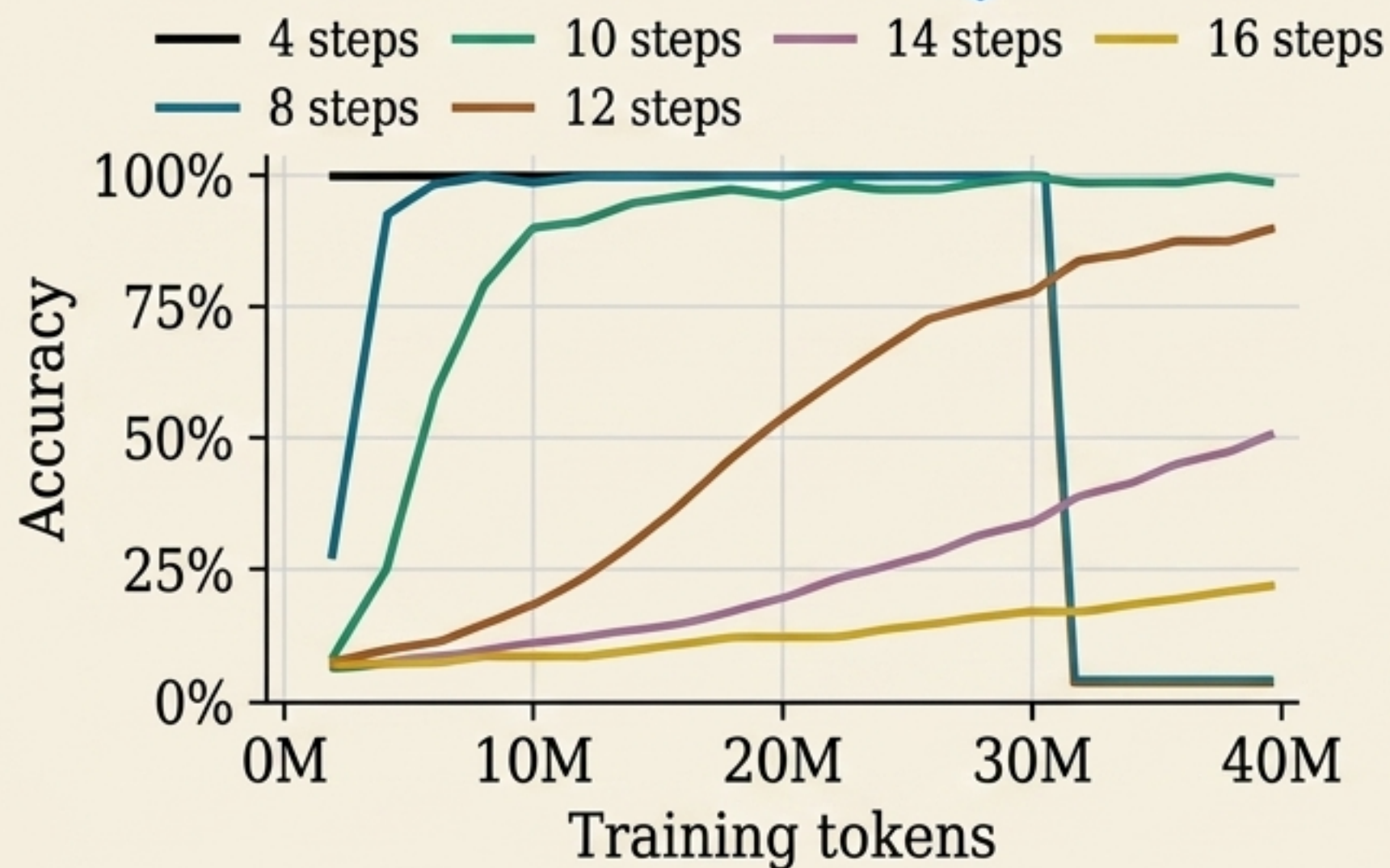
เปรียบเทียบสถาปัตยกรรม: ทำไม LLM Sleep จึงแตกต่าง?

คุณสมบัติ	Standard Transformer	Vanilla Hybrid	LLM Sleep (Proposed)
กลไกจัดเก็บ	KV Cache	KV + Fast Weights	KV + Fast Weights
ข้อจำกัดความยาว	ความจุต่ำ	ความจุสูง	ความจุสูง
การคิดวิเคราะห์ข้อมูล ที่ถูกลบ	ทำไม่ได้ (❌)	ต่ำ (⚠️)	สูงมาก (🟢)
ความเร็วในการ ตอบสนอง	ช้าเมื่อ Context ยาว (⚠️)	เร็ว (Single-pass) (🟢)	เร็ว (Single-pass) (🟢)

บททดสอบที่ 1: การจำลองลอจิกเชิงลึก (Cellular Automaton)

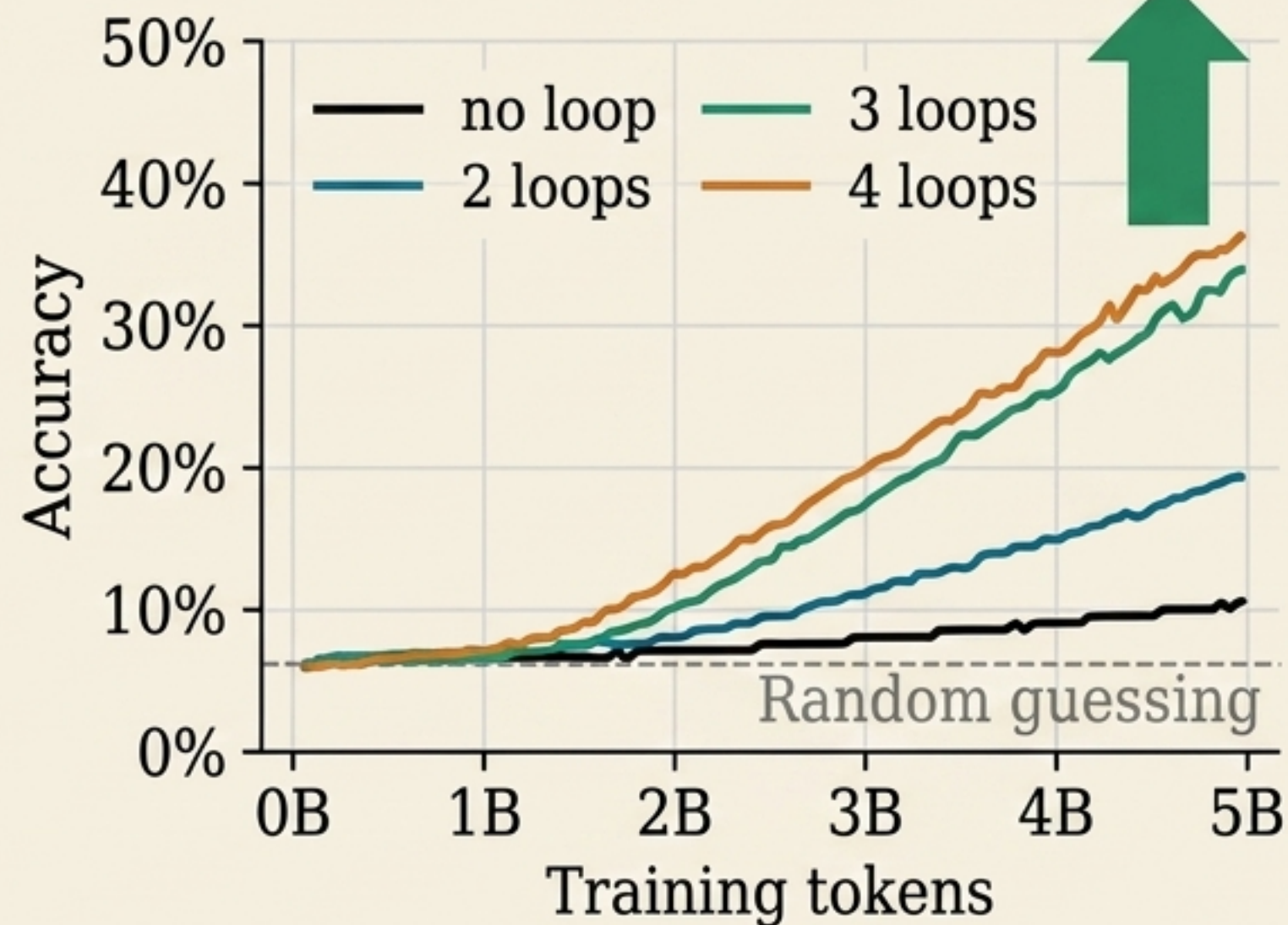
The Baseline Failure

เมื่อความลึกของโจทย์ (t) เพิ่มขึ้น สถาปัตยกรรม Hybrid ปกติล้มเหลวทันทีหลังถูก Evict

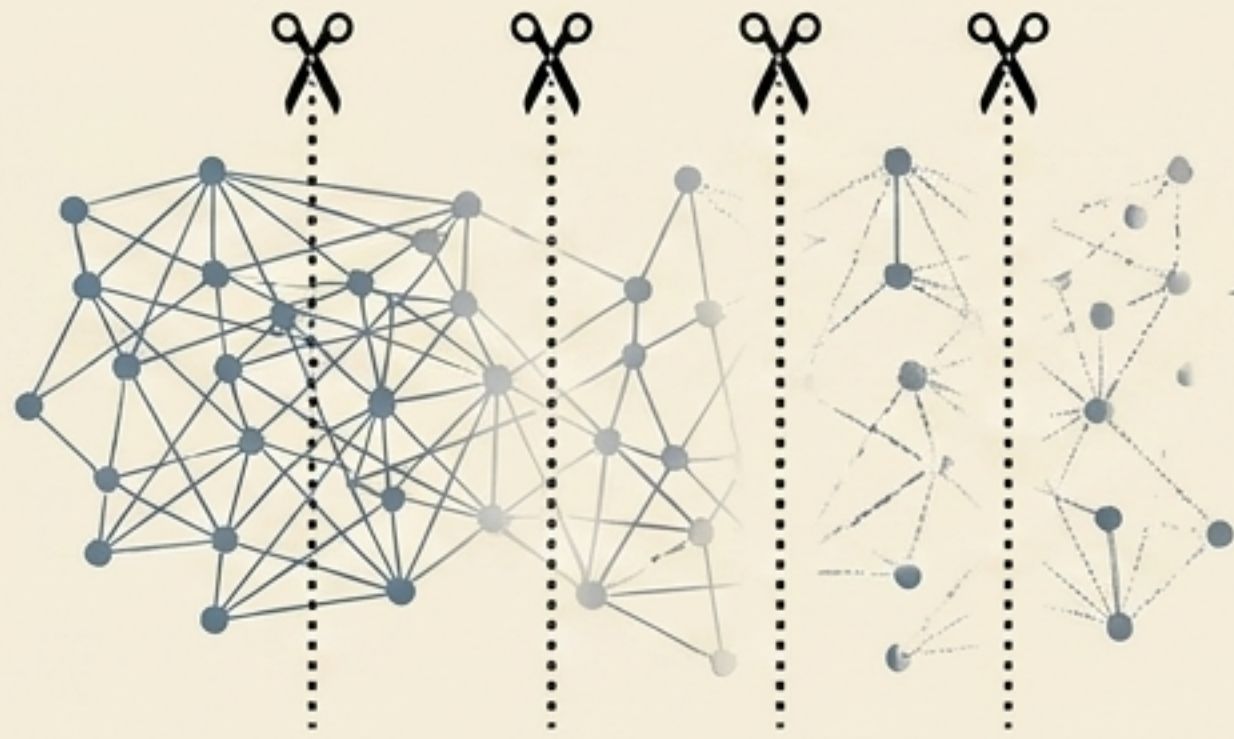


The Sleep Success

ผลลัพธ์: เพียงเพิ่มรอบการนอน (Sleep passes) ความแม่นยำในการคิดเชิงลึกก้าวกระโดดชนะ Random Guessing ได้สำเร็จ

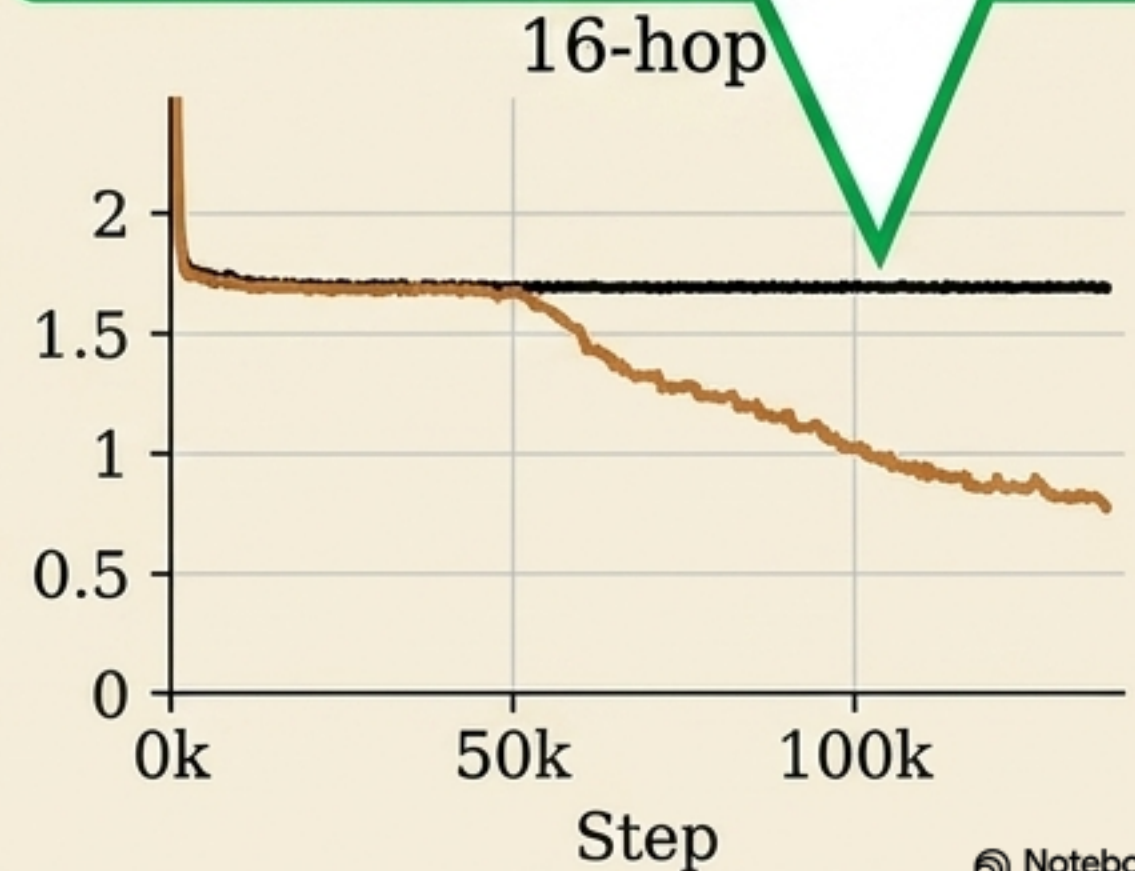
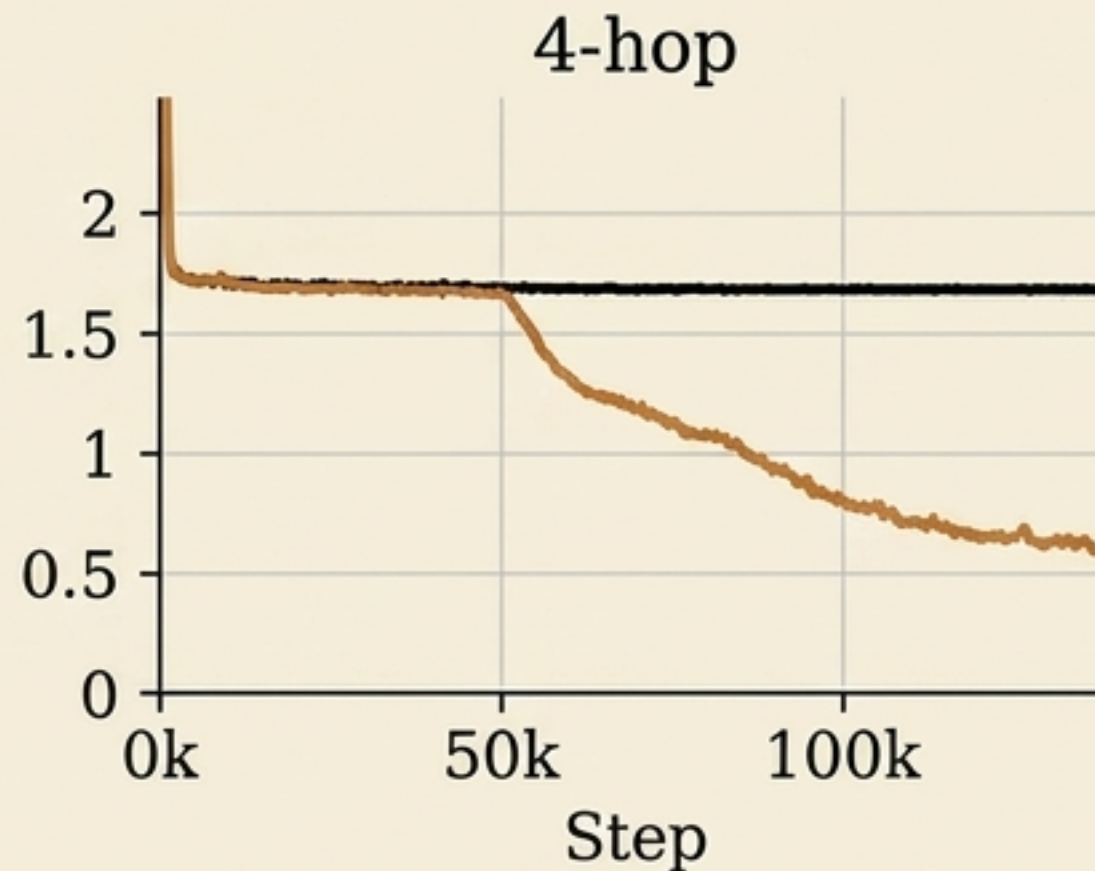
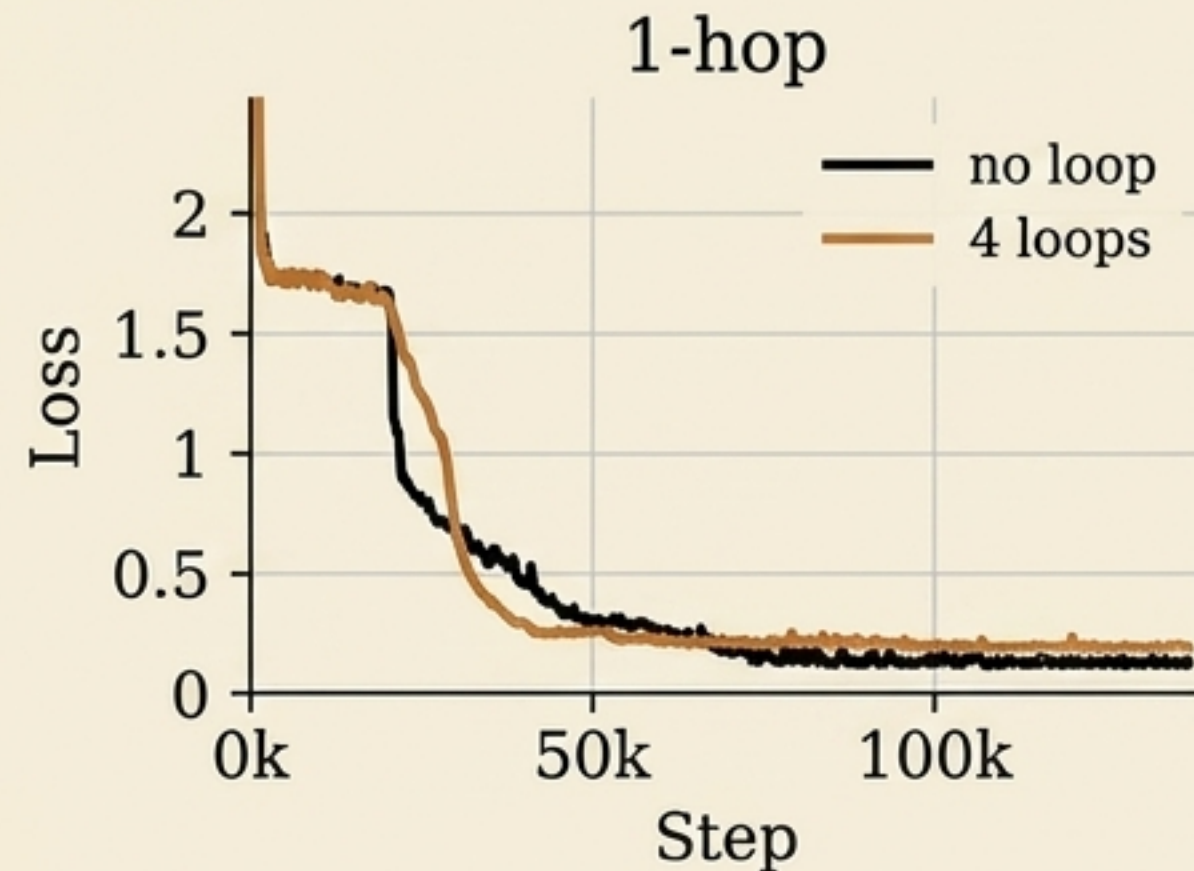


บททดสอบที่ 2: การดึงข้อมูลข้ามโหนด (Multi-Hop Knowledge Retrieval)

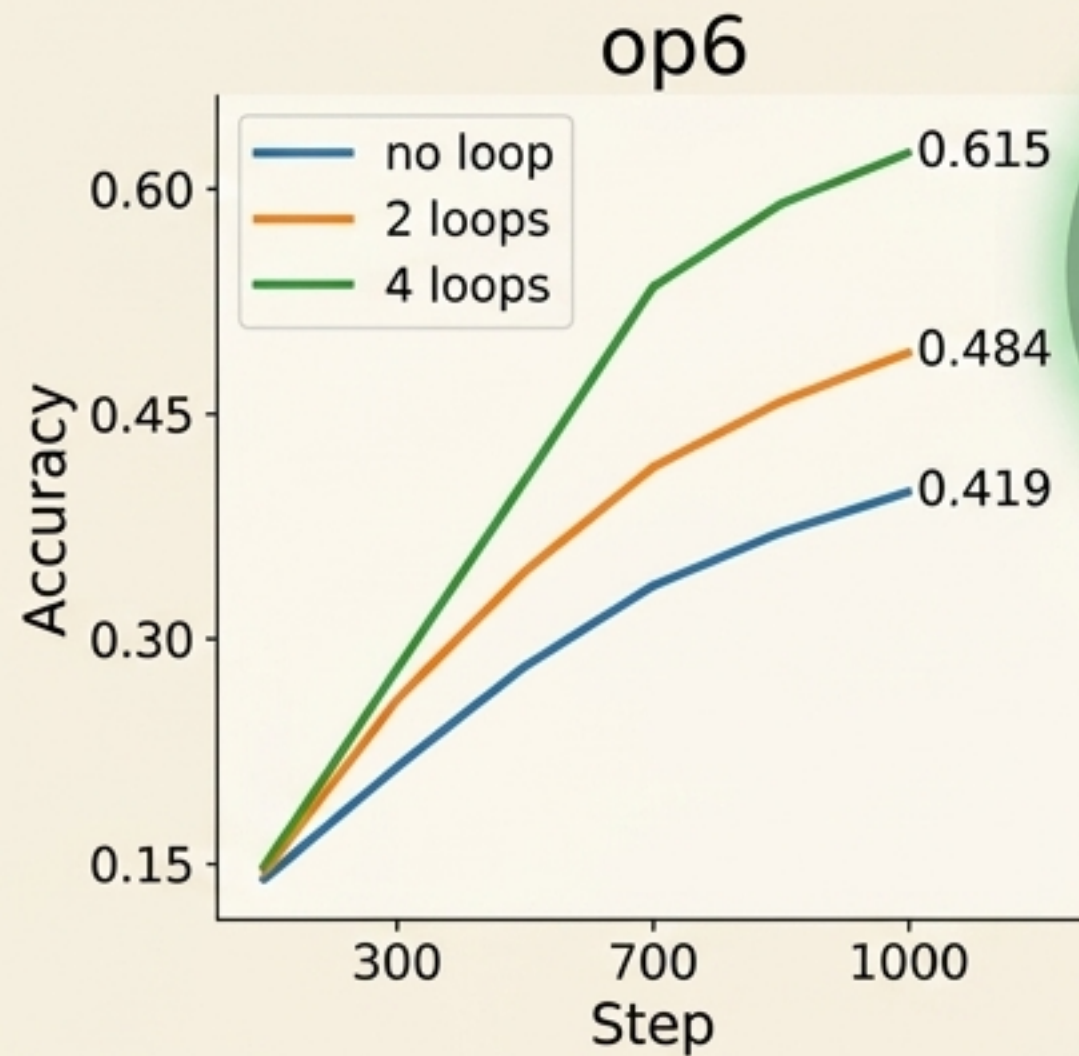


ความยาก: กราฟข้อมูลถูกหั่นแยกส่วนและลบออกจาก Cache
โมเดลต้องหาเส้นทางข้าม Node หลายทอด

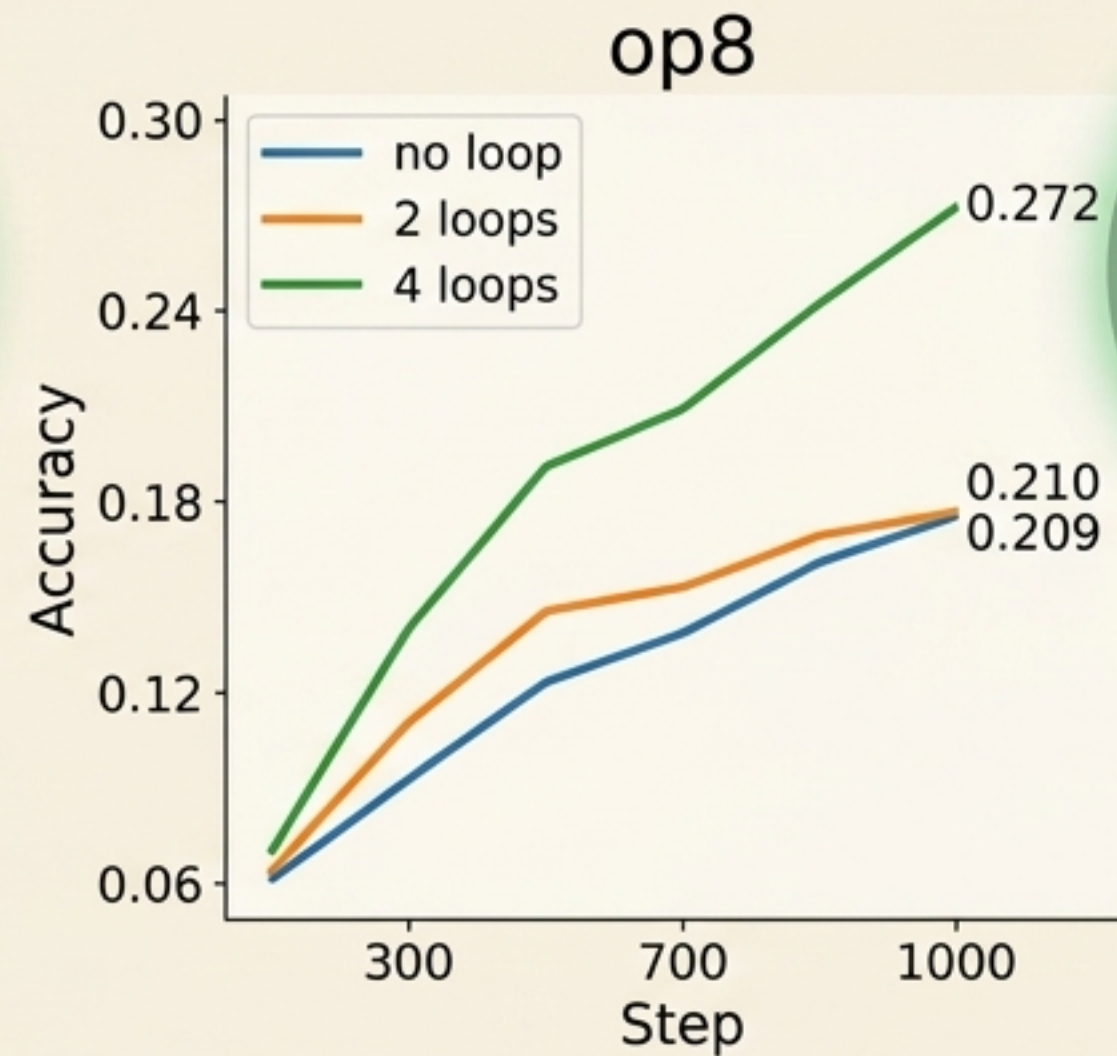
ในโจทย์ระดับยาก (16 ทอด)
โมเดลปกติแทบจะเรียนรู้ไม่ได้เลย (Stall)
แต่โมเดลที่มีการ Sleep 4 รอบ
สามารถทำความเข้าใจกราฟได้



บททดสอบบนข้อมูลจริง: คณิตศาสตร์ซับซ้อน (GSM-Infinite)



**+47%
Accuracy**



**+30%
Accuracy**

ทุกโมเดลทำได้ดีในโจทย์ง่าย แต่ในโจทย์ระดับยาก (6-8 ขั้นตอน)
การให้โมเดลนอนหลับช่วยเพิ่มความแม่นยำสูงสุดเกือบ 50%
ช่วยให้หลุดพ้นจากหลุมพรางของข้อมูลหลอก

ความยืดหยุ่นของสถาปัตยกรรม: Sliding-Window Eviction

Hard Eviction

Hard Eviction

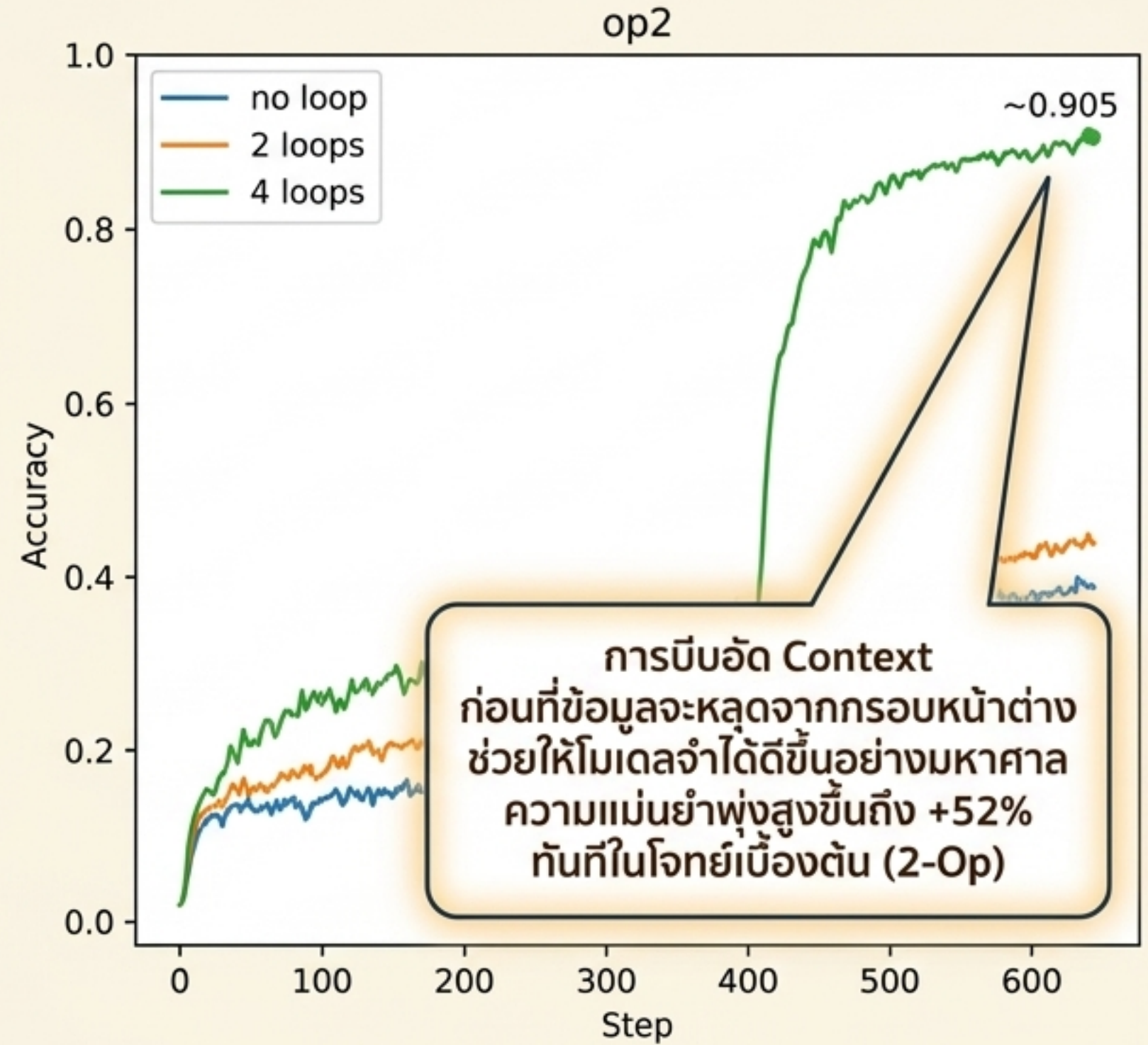


Sliding-Window

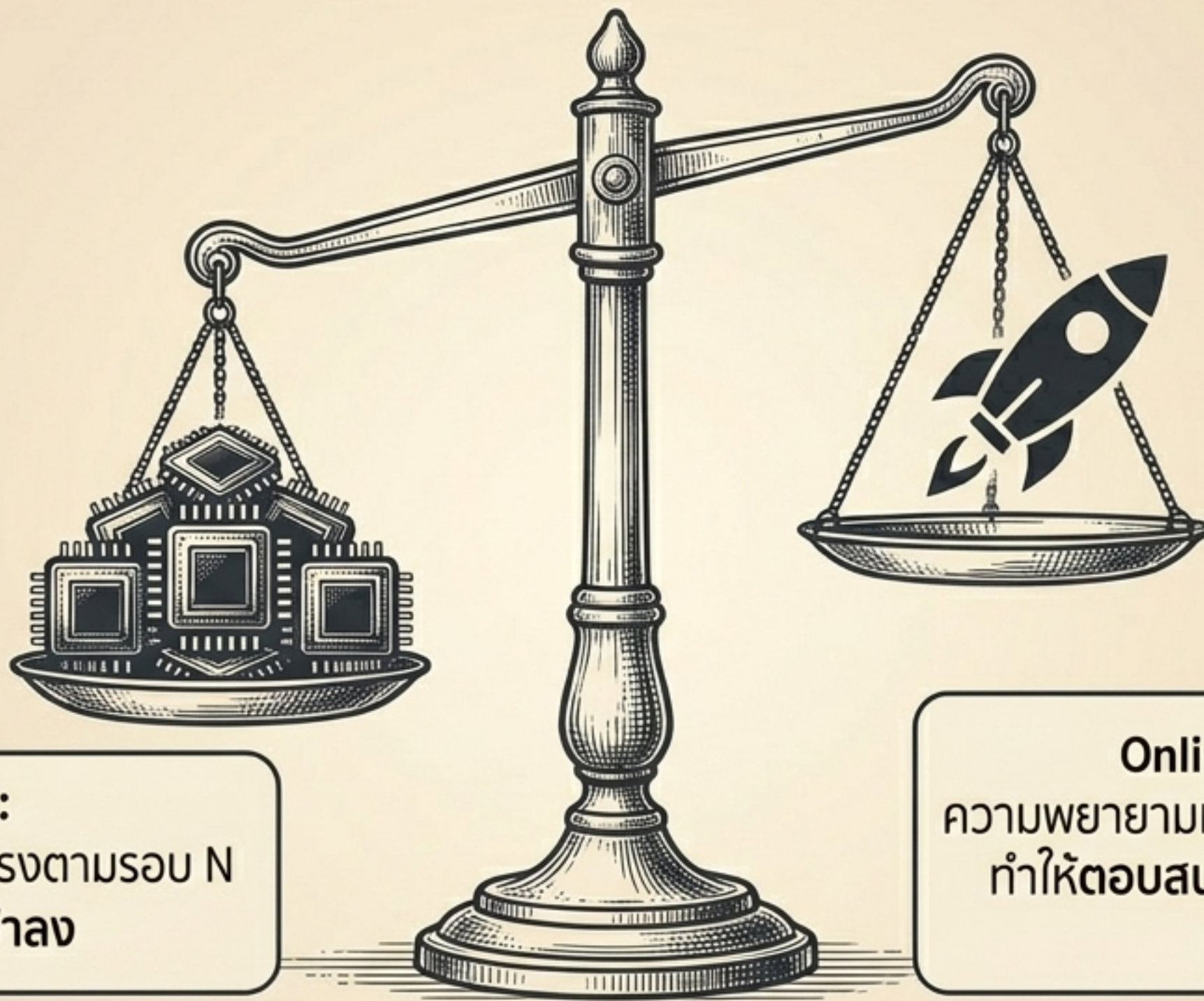
High-end editorial



Sliding Window Frame



สิ่งที่ต้องแลก: ต้นทุนการฝึกฝน vs ความเร็วในการใช้งานจริง



Training Phase:

ใช้ Compute เพิ่มขึ้นแบบเส้นตรงตามรอบ N
ทำให้การ Training ช้าลง

Online Inference Phase:

ความพยายามทั้งหมดถูกคำนวณและเก็บไว้แล้ว
ทำให้ตอบสนองเร็วเท่าเดิม (Single-pass)
สิ้นไหน ไม่หนัก

ยอมลงทุนคำนวณหนักตอน Offline เพื่อความเร็วและคุ้มค่าสุดขีดตอน Online

บทสรุป: รูปแบบใหม่ของการให้เหตุผล

Memory: Fast weights
จะไร้ประโยชน์ หากไม่มี
พลังคำนวณมากพอที่จะ
จัดระเบียบข้อมูล



Consolidation: การนอนหลับ
คือกุญแจสำคัญที่กลั่นกรอง
บริบทให้กลายเป็นความรู้ถาวร

Speed: แบนรับความหนักหน่วงไว้เบื้องหลัง
เพื่อรักษาความไวในการตอบสนอง

เปลี่ยน 'เวลาคิด' ให้กลายเป็น 'เวลาเรียนรู้' ล่วงหน้า